

UNIVERSITA' DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED
ELETTRICA E MATEMATICA APPLICATA



SOFTWARE ARCHITECTURE DESIGN

Music Playlist Manager

PROJECT WORK

First delivery: Pre-game

Group members:

Cognome	Nome	Matricola
Amoroso	Chiara	IE22700309
Contaldo	Luigi	IE22700111
D'Errico	Fabrizio	IE22700121
Esposito	Antonio	IE22700003

Anno accademico 2025/2026

Sommario

Obiettivo	3
Tools setup	3
Development platforms	3
Initial product backlog	4
Legenda	9
Definition of Done	10
Definition of the architecture of the solution	10
Model View Controller (MVC)	10
Note progettuali	10
Diagramma dei package	11
Diagramma UML	12
Design Pattern	12
Pattern Observer	12
Pattern Strategy	13
Pattern Command	14
Pattern Singleton	15
Tabella delle classi – Valutazione della coesione	16
Tabella dell'accoppiamento	17
Tabella riassuntiva – Coesione e Accoppiamento	18
User Interface MockUp	20
First sprint planning	20

Obiettivo

L'obiettivo di questo progetto consiste nel realizzare un'applicazione desktop per la creazione e gestione di playlist musicali, denominata **Music Playlist Manager**, che offra le seguenti funzionalità:

- Gestione (creazione, modifica, eliminazione e visualizzazione) delle tracce musicali, rappresentate da titolo, autore, durata, genere e anno di pubblicazione
- Organizzazione delle tracce in playlist identificate da un nome, con possibilità di aggiungere e rimuovere tracce
- Riproduzione simulata di tracce singole e playlist intere con controlli play, pausa e skip
- Tre modalità di riproduzione: sequenziale, shuffle e loop
- Modifica della playlist (aggiunta e rimozione di tracce) anche durante la riproduzione
- Annullamento (undo) delle operazioni di aggiunta e rimozione
- Visualizzazione nella home page delle tracce e playlist più frequentemente riprodotte
- Creazione automatica di playlist basate su genere o anno di pubblicazione
- Aggiunta di tag visivi (Preferito, Esplicito, Nuova Uscita) alle tracce per arricchire la visualizzazione e utilizzarli per la creazione automatica di playlist

Tools setup

Per facilitare la collaborazione, la gestione delle attività e il controllo del codice sorgente, il team ha adottato strumenti specifici che supportano in modo efficace il workflow di sviluppo. Di seguito sono indicati i principali tool utilizzati per l'organizzazione del progetto e la condivisione del codice:

- Github repo: https://github.com/fabriziodrr/SAD_musicplayer
- Trello: <https://trello.com/invite/b/6a0eb25333bf94cbb33b4b5c/ATTI964008534024cdfceb65d97725dbca2206F1E31E/sadprojectwork>

Development platforms

Lo sviluppo del progetto è stato supportato da un insieme di piattaforme e librerie che hanno facilitato l'implementazione, la realizzazione dell'interfaccia grafica e il testing. In particolare, sono stati adottati strumenti consolidati per lo sviluppo in Java, scelti per la loro affidabilità e integrazione con il workflow del team:

- NetBeans IDE
- JavaFX library per la GUI
- JUnit library per il testing

Initial product backlog

ID	TITLE	ITEM	PRIORITY	SP	ACCEPTANCE CRITERIA	STATUS
1	Aggiunta traccia al catalogo	Come utente voglio aggiungere un nuovo brano al catalogo inserendo titolo, autore, durata, genere e anno di pubblicazione, così da costruire la mia collezione musicale.	1	3	• Il sistema mostra un form con i campi: Titolo, Autore, Lunghezza, Genere, Anno • Non è possibile salvare una traccia se uno o più campi sono vuoti, viene mostrato un errore • La lunghezza accetta solo formato valido (mm:ss) • Al salvataggio la traccia appare immediatamente nel catalogo	Non ancora iniziato
2	Visualizzazione catalogo	Come utente voglio visualizzare tutte le tracce salvate nel catalogo con i loro attributi, così da avere sempre una visione completa della mia collezione.	1	2	• Il sistema mostra una lista con tutte le tracce salvate • Per ogni traccia sono visibili tutti i 5 attributi (Titolo, Autore, Durata, Genere, Anno) • Se il catalogo è vuoto viene mostrato il messaggio "Nessuna traccia presente"	Non ancora iniziato
3	Visualizzazione catalogo playlist	Come utente voglio visualizzare le playlist salvate con le tracce all'interno.	1	2	• Il sistema mostra una lista con tutte le playlist salvate • Per ogni Playlist sono visualizzabili tutte le tracce all'interno • Se il catalogo playlist è vuoto viene mostrato il messaggio "Nessuna playlist presente"	
4	Modifica traccia	Come utente voglio modificare i dati di un brano esistente, così da mantenere il catalogo sempre aggiornato.	1	2	• Selezionando una traccia si apre un form pre-compilato con i dati attuali • Modificando i dati e salvando le modifiche si	Non ancora iniziato

					riflettono immediatamente nel catalogo • Se l'utente annulla le modifiche apportate alla traccia mantiene i dati originali	
5	Eliminazione traccia	Come utente voglio eliminare una traccia dal catalogo, così da rimuovere contenuti che non voglio più.	1	3	• L'utente può selezionare una traccia e cliccare su "Elimina" • Il sistema chiede conferma prima dell'eliminazione definitiva • La traccia eliminata dal catalogo sparisce automaticamente da tutte le playlist in cui era presente	Non ancora iniziato
6	Creazione playlist	Come utente voglio creare una nuova playlist assegnandole un nome univoco, così da organizzare i miei brani in raccolte tematiche.	1	2	• L'utente può inserire il nome della nuova playlist • Il nome non può essere né vuoto né duplicato • La playlist creata è inizialmente vuota (0 tracce)	Non ancora iniziato
7	Aggiunta brani alla playlist	Come utente voglio aggiungere brani a una playlist, così da popolare le mie raccolte musicali.	1	3	• L'utente può selezionare una traccia dal catalogo e assegnarla a una playlist esistente • Più playlist possono contenere la stessa traccia	Non ancora iniziato
8	Rimozione brani dalla playlist	Come utente voglio rimuovere una traccia da una playlist senza eliminarla dal catalogo, così da modificare le mie raccolte senza perdere i brani.	1	2	• L'utente può selezionare una traccia all'interno di una playlist ed eliminarla • La traccia rimossa dalla playlist non viene eliminata dal catalogo generale	Non ancora iniziato
9	Controlli singola traccia	Come utente voglio fare play, pausa e skip di una singola traccia, così da	1	5	• Cliccando "Play" su una traccia il sistema simula l'avvio della riproduzione	Non ancora iniziato

		controllare la riproduzione simulata del brano			• Cliccando "Pause" il contatore si ferma • Cliccando di nuovo "Play" la riproduzione riparte esattamente da dove si era fermata • Cliccando "Skip" la riproduzione corrente si interrompe e passa alla traccia successiva	
10	Modalità di riproduzione playlist	Come utente voglio scegliere la modalità di riproduzione di una playlist tra sequenziale, loop e shuffle, così da personalizzare l'ordine di ascolto.	1	5	• Avviando una playlist il sistema riproduce in base alla modalità selezionata dall'utente • In modalità sequenziale le tracce vengono riprodotte in base alla loro posizione nella playlist • In modalità loop le tracce vengono riprodotte in ordine sequenziale e al termine della playlist si riparte dalla prima • In modalità shuffle le tracce vengono riprodotte in ordine casuale	Non ancora iniziato
11	Modalità di riproduzione singola traccia	Come utente voglio scegliere la modalità di riproduzione di una singola traccia tra sequenziale, loop e shuffle, così da controllare cosa viene riprodotto al termine del brano.	1	5	• Avviando una traccia il sistema riproduce in base alla modalità selezionata dall'utente • In modalità sequenziale al termine della traccia selezionata viene riprodotta la traccia successiva • In modalità loop al termine della traccia questa viene riprodotta nuovamente • In modalità shuffle al termine della	Non ancora iniziato

					traccia vengono riprodotte tracce in ordine casuale dalla playlist o dal catalogo	
12	Modifica playlist in esecuzione	Come utente voglio aggiungere o rimuovere tracce da una playlist mentre è in riproduzione, così da modificarla al volo senza interrompere l'ascolto.	1	5	• L'utente può aggiungere o rimuovere una traccia e la riproduzione del brano corrente continua senza interruzioni • Una traccia aggiunta viene posizionata in coda alla playlist • Se l'utente rimuove la traccia attualmente in esecuzione, al termine di quest'ultima viene riprodotto il brano successivo	Non ancora iniziato
13	Aggiornamenti o UI in tempo reale	Come utente voglio vedere il brano corrente, l'autore, il tempo trascorso e una barra di progresso aggiornati in tempo reale, così da sapere sempre cosa sta suonando e a che punto è la riproduzione.	1	5	• Il nome del brano corrente e l'autore sono visibili nella barra del lettore • La durata totale e il tempo trascorso sono visibili (es. "1:23 / 3:45") • Una barra di progresso mostra visivamente l'avanzamento in tempo reale	Non ancora iniziato
14	Undo operazioni	Come utente voglio annullare l'ultima operazione di aggiunta o rimozione di una traccia da una playlist, così da correggere errori accidentali senza doverli rifare manualmente.	2	3	• Undo (Inserimento): rimuove la traccia appena inserita (vale sia per l'inserimento della traccia nel catalogo che nella playlist) • Undo (Rimozione): ripristina la traccia appena eliminata dalla playlist (vale sia per la rimozione della traccia nel catalogo che nella playlist)	Non ancora iniziato

15	Statistiche home page	Come utente voglio vedere nella home page le tracce e le playlist riprodotte più frequentemente, così da ritrovare subito i miei contenuti preferiti.	2	3	• La home page mostra una sezione “Tracce più riprodotte” con le top 5 tracce • La home page mostra una sezione “Playlist più riprodotte” con le top 5 playlist • Le liste sono ordinate per numero di riproduzioni in ordine decrescente • Le liste si aggiornano automaticamente dopo ogni riproduzione	Non ancora iniziato
16	Playlist automatica per genere e anno	Come utente voglio generare automaticamente una playlist con tutte le tracce di un determinato genere/anno, così da creare raccolte tematiche senza aggiungerle manualmente.	2	5	• La playlist dedicata a quel genere viene creata automaticamente e popolata con i brani corrispondenti • Se a seguito di una cancellazione di una traccia non resta più nessun brano associato al genere o all'anno della playlist automatica, il sistema elimina automaticamente la playlist corrispondente.	Non ancora iniziato
17	Aggiunta tag visivi	Come utente voglio aggiungere tag visivi (Preferito, Esplicito, Nuova Uscita) a una traccia, così da arricchire la visualizzazione e identificare i brani a colpo d'occhio.	2	3	• Per ogni traccia sono disponibili tre tag: Preferito, Esplicito, Nuova Uscita • L'utente può attivare o disattivare un tag in fase di aggiunta o di modifica del brano • Una traccia può avere zero, uno o più tag contemporaneamente • I tag attivi sono visibili accanto al	Non ancora iniziato

					titolo della traccia nella lista • I tag vengono salvati e persistono al riavvio dell'applicazione	
18	Playlist automatica per tag	Come utente voglio generare automaticamente una playlist per tag visivo con le tracce corrispondenti, così da organizzare rapidamente i brani contrassegnati.	2	3	• Quando la rimozione o la modifica di un tag da una traccia comporta che una playlist automatica legata a quel tag rimanga senza brani, la playlist viene eliminata automaticamente dal sistema. • La playlist dedicata a quel tag viene creata automaticamente e popolata con i brani corrispondenti.	Non ancora iniziato
19	Riordino tracce nella playlist	Come utente voglio poter cambiare l'ordine delle tracce all'interno di una playlist affinché io possa personalizzare la sequenza di ascolto.	3	3	• L'utente deve poter spostare liberamente una traccia in una nuova posizione all'interno della playlist • Il nuovo ordine della playlist deve riflettersi immediatamente nell'interfaccia • L'utente deve poter riordinare i brani mentre la playlist è in esecuzione senza che la musica si interrompa	Non ancora iniziato

Legenda

- Per tutte le user story elencate, si assume che lo user sia sempre l'utente finale
- Le priorità sono da intendersi in ordine decrescente: ad esempio, gli items con priorità 1 hanno la massima priorità
- Gli story points sono stati assegnati utilizzando la sequenza di Fibonacci

Definition of Done

Di seguito sono elencati i criteri minimi che ogni user story deve soddisfare prima di poter essere considerata “Done” e inclusa nella versione finale del software:

- Ogni funzionalità del sistema è stata
 - Analizzata e progettata
 - Completamente implementata
 - Provata da un utente
- Il codice è pulito (senza stampe inutili o codice di test residuo) e adeguatamente commentato
- Ogni criterio di accettazione deve avere almeno un test case associato (quando applicabile)
- Tutti i test (sia JUnit che GUI) sono stati scritti, eseguiti e superati con successo
- Tutti i test di integrazione (sia JUnit che GUI) sono stati eseguiti, superati correttamente
- Il codice è stato caricato sul repository remoto e tutti i conflitti sono stati risolti
- Le implementazioni delle user story soddisfano TUTTI i criteri di accettazione
- Il codice è integrato nel branch principale su GitHub
- Le user story sono state peer-reviewed

Definition of the architecture of the solution

Per la progettazione del sistema abbiamo adottato il modello Object-Oriented, in quanto risulta essere il più efficace e intuitivo per la rappresentazione del dominio del problema relativo a questo tipo di progetto. Il dominio si compone di entità ben definite con comportamenti e responsabilità distinte, che si mappano direttamente su classi ad alta coesione. L'ereditarietà e il polimorfismo permettono di gestire in modo pulito le varianti di comportamento (modalità di riproduzione, operazioni reversibili) senza duplicare codice.

Model View Controller (MVC)

L'applicazione segue il pattern architetturale MVC (Model-View-Controller), naturalmente supportato da JavaFX tramite i file .fxml: Model (package modello): classi Traccia, Playlist, Catalogo e CatalogoPlaylist - rappresentano i dati e la logica di dominio, senza alcuna dipendenza dalla UI. View (file .fxml): definisce la struttura grafica dell'interfaccia utente in modo dichiarativo, separata dalla logica. Controller (package ui): PrimaryController gestisce gli eventi utente, aggiorna il Model e riflette i cambiamenti sulla View, senza contenere logica di dominio. Questa separazione garantisce che la logica di business rimanga testabile indipendentemente dalla GUI.

Note progettuali

- Le classi sono state progettate per rendere il codice modulare e facilmente estendibile.
- **PrimaryController** implementa il ruolo di Controller nel pattern MVC: gestisce gli eventi della View (file .fxml) e delega le operazioni alle classi del Model e dei servizi, senza contenere logica di dominio.

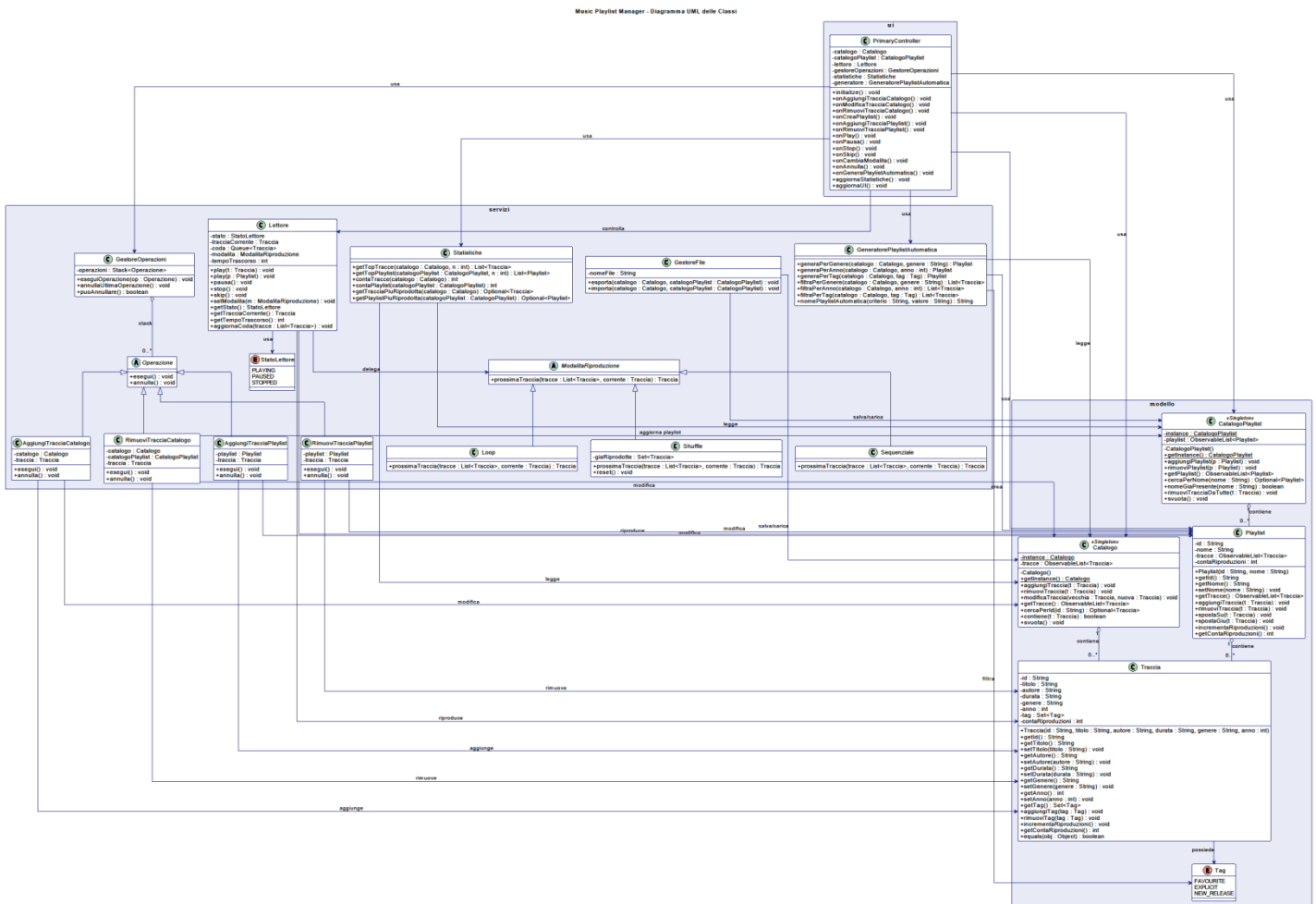
- **Playlist** contiene una `ObservableList` e rappresenta una raccolta personalizzata creata dall'utente. Consente un ordine personalizzabile, la presenza di tracce duplicate e l'aggiornamento automatico della UI durante la riproduzione.
- I metodi `svuota()` di **Catalogo** e **CatalogoPlaylist** azzerano completamente le rispettive collezioni. Sono utilizzati principalmente durante l'importazione da file, per ricaricare uno stato pulito prima di popolare le liste con i dati letti da **GestoreFile**.
- **Statistiche** contiene le operazioni di sola lettura per calcolare le tracce e le playlist più riprodotte. Riceve **Catalogo** e **CatalogoPlaylist** come sorgenti dati, senza modificarli. I metodi `getTopTracce(n)` e `getTopPlaylist(n)` restituiscono le prime N tracce/playlist per numero di riproduzioni, usati nella sezione "Top 5" della home page. I metodi `getTracciaPiuRiprodotta()` e `getPlaylistPiuRiprodotta()` restituiscono il singolo elemento più riprodotto e sono disponibili per widget o sezioni dedicate della UI che richiedono un unico elemento in evidenza.
- **GeneratorePlaylistAutomatica** crea playlist automatiche filtrando le tracce per genere, anno o tag. Restituisce una **Playlist** generata, lasciando al controller o a **CatalogoPlaylist** la decisione di salvarla. Il metodo `nomePlaylistAutomatica(criterio, valore)` genera automaticamente il nome da assegnare alla playlist prodotta, componendolo a partire dal criterio di filtro e dal valore usato (es. "Rock - 2020" per genere Rock e anno 2020).
- **GestoreFile** si occupa dell'esportazione e dell'importazione persistente di **Catalogo** e **CatalogoPlaylist** su file (formato JSON o serializzazione Java). L'attributo `nomeFile` contiene il percorso del file da salvare/caricare.
- **Lettore** gestisce la riproduzione simulata mantenendo lo stato corrente (PLAYING, PAUSED, STOPPED). Delega a **ModalitaRiproduzione** il calcolo della prossima traccia tramite `prossimaTraccia()`: il risultato viene inserito nella Queue interna, che **Lettore** consuma sequenzialmente.
- **ModalitàRiproduzione** è una classe astratta (pattern Strategy) con tre sottoclassi concrete: **Sequenziale** (avanza in ordine), **Shuffle** (ordine casuale), **Loop** (ripetizione ciclica). La modalità attiva è impostabile a runtime senza modificare **Lettore**.
- **Operazione** è una classe astratta (pattern Command) con le sottoclassi concrete **AggiungiTracciaCatalogo** e **RimuoviTracciaCatalogo**, che ridefiniscono `esegui()` e `annulla()`. Analogamente esistono **AggiungiTracciaPlaylist** e **RimuoviTracciaPlaylist** per le operazioni sulle playlist. Questo consente di aggiungere nuove operazioni reversibili per semplice estensione senza modificare **GestoreOperazioni**.

Diagramma dei package

Package	Descrizione
UI	Contiene PrimaryController e le classi responsabili dell'interfaccia utente. Dipende da modello e servizi, ma non viceversa.
Modello	Contiene le classi di dominio pure (Traccia , Playlist , Catalogo , CatalogoPlaylist). Nessuna dipendenza dalla UI o dai servizi applicativi.
Servizi	Contiene le classi di logica applicativa (Lettore , GestoreFile , GestoreOperazioni , ModalitaRiproduzione , Operazione , Statistiche , GeneratorePlaylistAutomatica). Dipende solo da modello.

Diagramma UML

E' stata predisposta una bozza del diagramma UML delle classi:



Design Pattern

- **Observer** - Catalogo, CatalogoPlaylist e Playlist usano ObservableList di JavaFX: ogni modifica notifica automaticamente la View senza accoppiamento esplicito.
- **Strategy** - ModalitaRiproduzione è una classe astratta le cui sottoclassi concrete (Sequenziale, Shuffle, Loop) implementano il `prossimaTraccia()`, permettendo di cambiare algoritmo a runtime senza modificare Lettore.
- **Command** - Operazione è una classe astratta che incapsula le operazioni reversibili; GestoreOperazioni mantiene uno stack di comandi per supportare l'undo.
- **Singleton** - Catalogo e CatalogoPlaylist vengono istanziati una sola volta nell'applicazione, garantendo un unico punto di accesso rispettivamente alla collezione di tracce e alla collezione di playlist.

Pattern Observer

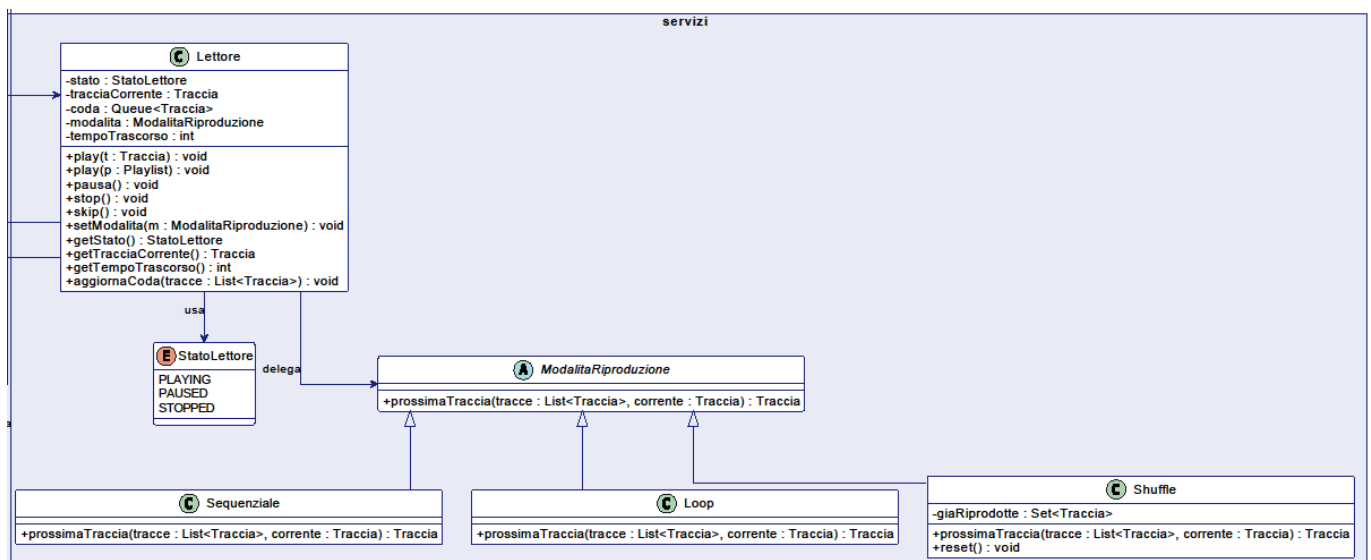
In questa parte del diagramma è stato implementato il *Behavioral Pattern* Observer (tramite ObservableList di JavaFX) in quanto:

- Permette di aggiornare automaticamente la View ogni volta che le collezioni di tracce o playlist vengono modificate, senza accoppiamento esplicito tra il Model e il PrimaryController
- Permette di eliminare la necessità di aggiornare manualmente la UI dopo ogni operazione di aggiunta, rimozione o modifica
 - Se una traccia viene aggiunta tramite `Catalogo.aggiungiTraccia()`, la `ObservableList` notifica automaticamente tutti i componenti JavaFX ad essa collegati (es. `TableView`) senza che `Catalogo` conosca l'esistenza della View

`Catalogo`, `CatalogoPlaylist` e `Playlist` fungono da **Subject** → contengono ciascuna una `ObservableList` che notifica automaticamente i componenti osservatori ad ogni modifica.

Il `PrimaryController` funge da **Observer** → si registra sulle `ObservableList` tramite binding JavaFX e aggiorna la UI in tempo reale senza polling.

Pattern Strategy



In questa parte del diagramma è stato implementato il **Behavioral Pattern Strategy** in quanto:

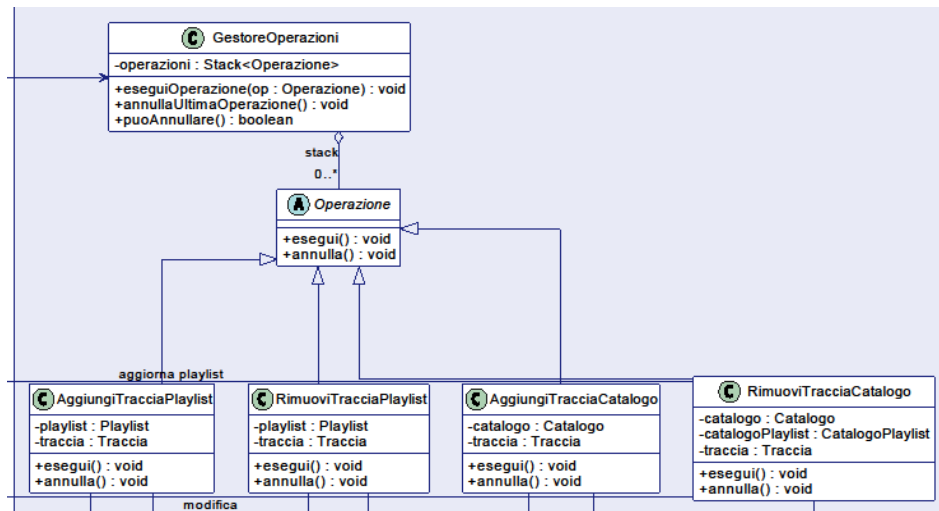
- Permette di separare la logica di selezione della prossima traccia dalla classe `Lettore`, aderendo al principio di *single responsibility*
- Permette di cambiare l'algoritmo di riproduzione a runtime senza modificare il `Lettore`
 - Se la logica di una modalità dovesse cambiare (es. `Shuffle` con nuovi criteri di casualità), questa può essere modificata all'interno della sottoclasse corrispondente senza impattare il resto del sistema
- Permette di aggiungere nuove modalità di riproduzione per semplice estensione, creando una nuova sottoclasse di `ModalitaRiproduzione` senza toccare il codice esistente, aderendo al principio *Open/Closed*

La classe `Lettore` funge da **Context** → mantiene un riferimento a `ModalitaRiproduzione` e delega il calcolo della traccia successiva tramite `prossimaTraccia()`.

La classe astratta `ModalitaRiproduzione` funge da **Strategy** → definisce l'interfaccia comune tramite il metodo `prossimaTraccia(tracce, corrente)`.

Le classi Sequenziale, Loop e Shuffle fungono da **ConcreteStrategy** → ognuna implementa prossimaTraccia() con il proprio algoritmo. In particolare Shuffle mantiene internamente un Set<Traccia> per evitare ripetizioni consecutive e offre un metodo reset() per il reinizio del ciclo.

Pattern Command



In questa parte del diagramma è stato implementato il **Behavioral Pattern Command** in quanto:

- Permette di separare la logica di esecuzione delle operazioni dalla classe che le richiede, aderendo al principio di *single responsibility*
- Permette di avere il controllo centralizzato sull'esecuzione e l'annullamento delle operazioni all'interno di **GestoreOperazioni**
 - Se una nuova operazione reversibile dovesse essere aggiunta (es. rinomina playlist), questa può essere implementata creando una nuova sottoclasse di **Operazione** senza modificare **GestoreOperazioni** né il resto del sistema
- Permette di implementare la funzionalità di undo in modo strutturato ed estendibile, grazie allo `Stack<Operazione>` mantenuto da **GestoreOperazioni**

La classe **GestoreOperazioni** funge da **Invoker** → mantiene uno `Stack<Operazione>` e gestisce i metodi `+eseguiOperazione()`, `+annullaUltimaOperazione()` e `+puoAnnullare()` senza conoscere i dettagli concreti di ogni operazione.

La classe astratta **Operazione** funge da **Command** → definisce l'interfaccia comune tramite i metodi astratti `+esegui()` e `+annulla()`.

Le classi **AggiungiTracciaCatalogo**, **RimuoviTracciaCatalogo**, **AggiungiTracciaPlaylist** e **RimuoviTracciaPlaylist** fungono da **ConcreteCommand** → ognuna implementa `+esegui()` e `+annulla()` con la propria logica specifica. In particolare **RimuoviTracciaCatalogo** memorizza anche il riferimento a **CatalogoPlaylist** per garantire la corretta propagazione della rimozione in tutte le playlist.

Pattern Singleton



In questa parte del diagramma è stato implementato il *Creational Pattern Singleton* in quanto:

- Permette di garantire che esistano istanze uniche di **Catalogo** e **CatalogoPlaylist** nell'intera applicazione, evitando incongruenze nei dati dovute a istanze multiple
- Permette di avere punti di accesso globali e centralizzati alle collezioni condivise tra **PrimaryController** e tutti i servizi (**GestoreFile**, **Statistiche**, **GeneratorePlaylistAutomatica**, **GestoreOperazioni**)
 - Se **PrimaryController** accede a **Catalogo** o a **CatalogoPlaylist** in più punti del codice, otterrà sempre la stessa istanza tramite `Catalogo.getInstance()` senza rischio di dati duplicati o non sincronizzati

Sia **Catalogo** che **CatalogoPlaylist** implementano il Singleton tramite:

- L'attributo statico privato `-instance` che mantiene l'unica istanza
- Il costruttore privato che impedisce la creazione di istanze dall'esterno
- Il metodo statico pubblico `+getInstance()` che restituisce sempre la stessa istanza

Tabella delle classi – Valutazione della coesione

Classe	Descrizione	Coesione
Catalogo	Gestisce la collezione generale delle tracce tramite ObservableList<Traccia>	FUNZIONALE - tutti i metodi (aggiungiTraccia(), rimuoviTraccia(), modificaTraccia(), getTracce()) contribuiscono allo stesso obiettivo: gestire il catalogo delle tracce.
CatalogoPlaylist	Gestisce la collezione generale delle playlist tramite ObservableList. Non gestisce le tracce interne di una singola playlist.	FUNZIONALE - tutti i metodi (aggiungiPlaylist(), rimuoviPlaylist(), getPlaylist(), nomeGiaPresente()) operano sulla raccolta globale delle playlist.
Traccia	Contiene le informazioni della singola traccia (titolo, autore, durata, genere, anno) e i tag visivi associati.	FUNZIONALE - i metodi (get, set, equals, compareTo, costruttore) operano esclusivamente su un oggetto Traccia.
Playlist	Contiene una lista ordinata di tracce e i metodi per gestirla (aggiunta, rimozione, riordino).	FUNZIONALE - tutti i metodi operano sullo stesso oggetto Playlist e ne gestiscono lo stato interno.
Statistiche	Calcola dati riepilogativi di sola lettura, come le tracce e le playlist più riprodotte.	COMUNICAZIONALE - i metodi lavorano sugli stessi dati persistenti: tracce e playlist.
Generatore Playlist automatica	Crea playlist automatiche filtrando le tracce per genere, anno o tag	FUNZIONALE - tutti i metodi partecipano allo stesso compito: costruire una nuova Playlist sulla base di un criterio specifico (genere, anno o tag).
GestoreFile	Salvataggio e lettura di Catalogo e Catalogo Playlist su file persistente, mediante esporta() e importa().	COMUNICAZIONALE - i metodi lavorano sugli stessi dati persistenti: tracce e playlist.
Lettore	Gestisce la riproduzione simulata con play(), pausa(), skip(), usando una Queue per le prossime tracce e delegando la selezione a Modalita Riproduzione.	FUNZIONALE - tutti i metodi lavorano sullo stato di riproduzione corrente.
Modalità riproduzione	Classe astratta (Strategy) con sottoclassi Sequenziale, Shuffle, Loop. Ridefinisce prossimaTraccia().	COMUNICAZIONALE - i metodi lavorano sugli stessi dati (lista tracce e traccia corrente).
Operazione	Classe astratta (Command) per le operazioni reversibili. Sottoclassi concrete: Aggiungi	COMUNICAZIONALE - entrambi i metodi operano sugli stessi dati.

	TracciaCatalogo, Rimuovi TracciaCatalogo, Aggiungi TracciaPlaylist, Rimuovi TracciaPlaylist. Ridefinisce esegui() e annulla().	
GestoreOperazioni	Mantiene lo stack delle ultime operazioni per consentire l'undo.	FUNZIONALE - tutti i metodi operano sullo stack delle operazioni.
PrimaryController	Gestisce gli eventi utente e coordina model e servizi.	FUNZIONALE - ogni metodo gestisce un evento dell'interfaccia e delega la logica a classi specializzate.

Tabella dell'accoppiamento

Relazione	Tipo	Motivazione
Traccia <-> Catalogo	PER DATI	Catalogo memorizza e restituisce oggetti Traccia; non dipende dalla logica interna della traccia, ma solo dalla sua interfaccia pubblica.
Traccia <-> Playlist	PER DATI	Playlist riceve in input solo l'oggetto Traccia da aggiungere, rimuovere o riordinare, senza dipendere dal catalogo generale.
Playlist <-> Catalogo Playlist	PER DATI	CatalogoPlaylist conserva oggetti Playlist e ne richiama solo i metodi pubblici necessari alla gestione della raccolta.
Catalogo <-> Gestore File	PER DATI	GestoreFile salva e carica la lista delle tracce tramite l'interfaccia pubblica di Catalogo, senza gestire logica di dominio
CatalogoPlaylist <-> GestoreFile	PER DATI	GestoreFile salva e carica la lista delle playlist tramite l'interfaccia pubblica di CatalogoPlaylist.
Catalogo <-> Statistiche	PER DATI	Statistiche legge le tracce dal catalogo per calcolare le top-N, senza modificare la lista e senza conoscere dettagli implementativi.
CatalogoPlaylist <-> Statistiche	PER DATI	Statistiche legge le playlist per calcolare le più riprodotte, usando solo dati pubblici.
Catalogo <-> GeneratorePlaylist Automatica	PER DATI	Il generatore legge le tracce dal catalogo, applica un filtro e produce una nuova Playlist, senza modificare direttamente il catalogo.

GeneratorePlaylist Automatica <-> Playlist	PER DATI	Il generatore costruisce una playlist usando oggetti Traccia; la persistenza della playlist resta responsabilità del chiamante.
Catalogo <-> Primary Controller	PER DATI	Il controller richiama solo metodi pubblici come aggiungiTraccia(), rimuoviTraccia() e getTracce().
CatalogoPlaylist <-> PrimaryController	PER DATI	Il controller gestisce le operazioni sulle playlist tramite metodi pubblici del catalogo playlist.
Playlist <-> Primary Controller	PER DATI	Al controller vengono passati solo gli oggetti Playlist necessari alla visualizzazione e alla gestione delle operazioni utente.
Lettore <-> Modalità Riproduzione	PER DATI	Lettore delega la scelta della traccia successiva passando solo la lista delle tracce e la traccia corrente; nessun dettaglio implementativo è esposto. La modalità è sostituibile a runtime (Strategy).
GestoreOperazioni <-> Operazione	PER DATI	GestoreOperazioni memorizza le operazioni nello stack senza conoscerne i dettagli concreti, richiamando solo l'interfaccia esegui() e annulla() definita dalla superclasse
Lettore <-> Primary Controller	PER DATI	Il controller richiama i metodi pubblici play(), pausa(), stop(), skip() per controllare la riproduzione, e getStato() e getTempoTrascorso() per aggiornare la barra di progresso e lo stato della UI in tempo reale, senza accedere allo stato interno del Lettore.

Tabella riassuntiva – Coesione e Accoppiamento

Classe	Package	Coesione	Accoppiamento
Traccia	Modello	FUNZIONALE - tutti i metodi operano esclusivamente su un singolo oggetto Traccia. I tag visivi sono attributi della stessa entità.	Per dati con Catalogo, Playlist e GeneratorePlaylist Automatica: vengono scambiati solo oggetti Traccia.
Catalogo	Modello	FUNZIONALE - gestisce solo la	Per dati con Gestore File, Statistiche, GeneratorePlaylist

		collezione generale delle tracce e le relative operazioni CRUD.	Automatica e Primary Controller: espone solo metodi pubblici e liste osservabili.
Catalogo Playlist	Modello	FUNZIONALE - gestisce solo la collezione generale delle playlist e il controllo di unicità dei nomi.	Per dati con Gestore File, Statistiche e PrimaryController: scambia oggetti Playlist senza dipendere dalla loro implementazione interna.
Playlist	Modello	FUNZIONALE - tutti i metodi (aggiungiTraccia, rimuoviTraccia, spostaSu, spostaGiu) gestiscono lo stato interno della playlist e la sua ObservableList	Per dati con Catalogo Playlist, Primary Controller e GeneratorePlaylist Automatica: riceve/passa solo oggetti Playlist e Traccia.
Statistiche	Servizi	FUNZIONALE - tutti i metodi producono riepiloghi di sola lettura, come top tracce e top playlist.	Per dati con Catalogo e CatalogoPlaylist: legge dati tramite metodi pubblici senza modificarli.
Generatore Playlist Automatica	Servizi	FUNZIONALE - tutti i metodi filtrano tracce e costruiscono playlist automatiche.	Per dati con Catalogo e Playlist: legge tracce e restituisce una playlist generata.
GestoreFile	Servizi	COMUNICAZIONALE - esporta() e importa() operano sugli stessi dati persistenti: tracce e playlist.	Per dati con Catalogo e CatalogoPlaylist: salva/carica liste tramite metodi pubblici.
Lettore	Servizi	FUNZIONALE - tutti i metodi (play, pausa, stop, skip) lavorano sullo stato di riproduzione e sulla Queue interna.	Per dati con Modalità Riproduzione (delega via Strategy passando solo lista tracce + traccia corrente) e con Primary Controller (chiama solo metodi pubblici senza accedere allo stato interno).
Modalità Riproduzione	Servizi	COMUNICAZIONALE - le sottoclassi Sequenziale, Shuffle, Loop ridefiniscono prossimaTraccia() operando sugli stessi input (lista tracce, traccia corrente).	Per dati con Lettore: il pattern Strategy isola l'algoritmo; la modalità è intercambiabile a runtime senza modificare il Lettore.
Operazione	Servizi	COMUNICAZIONALE - ogni sottoclasse concreta (Aggiungi TracciaCatalogo, Rimuovi TracciaCatalogo, etc.) ridefinisce esegui() e annulla() sugli stessi dati.	Per dati con Gestore Operazioni: lo stack memorizza comandi astratti e chiama solo l'interfaccia esegui()/annulla() (pattern Command).
Gestore Operazioni	Servizi	FUNZIONALE - tutti i metodi (eseguiOperazione, annullaUltimaOperazione) operano esclusivamente sullo stack LIFO delle operazioni.	Per dati con Operazione: interagisce solo tramite l'interfaccia astratta, senza conoscere le implementazioni concrete.
Primary Controller	UI	FUNZIONALE - ogni metodo gestisce un evento UI specifico (click su pulsante, selezione tabella) delegando la logica a modello e servizi.	Per dati con Catalogo, CatalogoPlaylist, Playlist, Lettore, Statistiche e GeneratorePlaylist Automatica: il controller invoca solo metodi

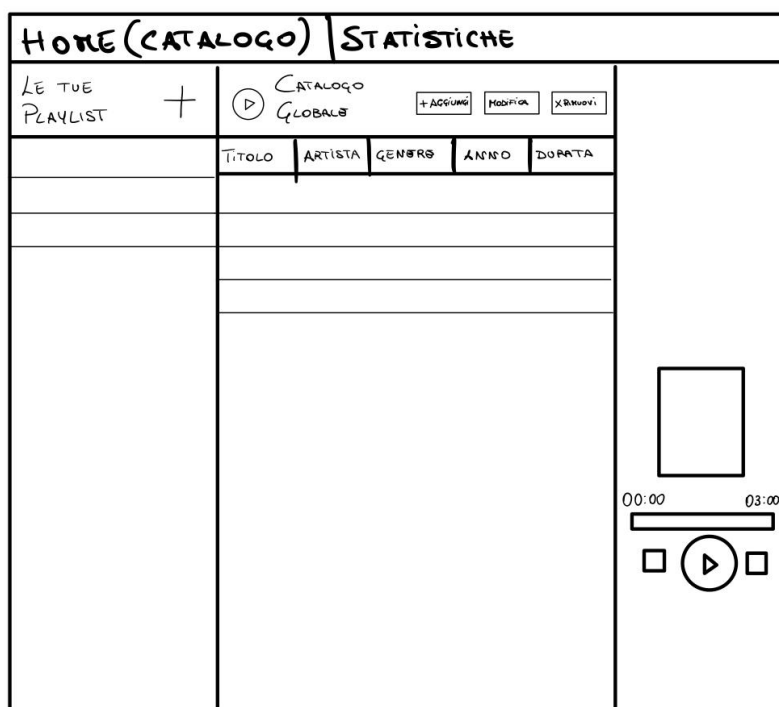
			pubblici e non contiene logica di dominio.
--	--	--	--

Legenda coesione: Funzionale = massima coesione (ogni metodo contribuisce a un unico compito ben definito); **Comunicazionale** = metodi che operano sugli stessi dati di input/output; **Procedurale** = metodi eseguiti in sequenza come parte di una stessa operazione.

Legenda accoppiamento: Per dati = forma più bassa di accoppiamento; i moduli comunicano scambiandosi solo dati semplici o oggetti, senza dipendere dalla logica interna altrui. L'uso dei pattern Strategy, Command e Singleton contribuisce a mantenere basso l'accoppiamento in tutto il sistema.

User Interface MockUp

Quella riportata di seguito rappresenta una bozza preliminare dell'interfaccia utente che sarà alla base del sistema da sviluppare negli sprint successivi.



First sprint planning

Nel 1° Sprint il team ha deciso di implementare le funzionalità fondamentali per consentire all'utente di iniziare a utilizzare l'applicazione in modo efficace. L'obiettivo è costruire una base solida e interattiva per la gestione della libreria musicale: l'utente dovrà essere in grado di aggiungere, modificare ed eliminare tracce, creare e popolare playlist, e avviare la riproduzione simulata di una singola traccia con i relativi controlli di play, pausa e skip.

Abbiamo selezionato le funzionalità essenziali in base alla velocità iniziale stimata del team, pari a **22 Story Points (SP)**. Questo ci ha permesso di includere nel backlog dello sprint una serie di user

stories coerenti tra loro e ad alto valore per l'utente, mantenendo al tempo stesso un carico di lavoro sostenibile per i 4 membri del team.

Le user stories selezionate coprono parzialmente le funzionalità ad **alta priorità** (priorità 1) relative alla gestione della libreria e delle playlist, nonché i controlli base di riproduzione simulata. Le funzionalità più avanzate - come le modalità di riproduzione (shuffle, loop o sequenziale), la modifica della playlist durante il playback e l'aggiornamento dell'UI in tempo reale - sono state volutamente rimandate allo Sprint 2 in quanto richiedono come prerequisito le funzionalità di base implementate in questo sprint.

Questo sprint mira quindi a consegnare una **prima versione funzionante** del software, già dotata di una gamma di strumenti coerente e utile, sulla quale costruire le funzionalità avanzate nei sprint successivi.